

Dynamic Parameter Identification of the 6-DoF Manipulator

0.1 Dynamic Modeling and Base Parameter Formulation

0.1.1 Inverse Dynamics via Newton-Euler Formulation

The primary purpose of modeling the robot dynamics is to establish a foundation for real-time, high-precision control, where an accurate dynamic model and precise dynamic parameters are essential prerequisites. The rigid-body dynamics of a serial manipulator can be mathematically expressed utilizing either the Lagrange method or the Newton-Euler method. The Newton-Euler formulation, which establishes the dynamic equations from the perspective of force equilibrium, is widely adopted due to its high computational efficiency. Furthermore, its iterative properties are highly conducive to software implementation and numerical simulation.

For an n -degree-of-freedom (DoF) serial manipulator, such as the 6-DoF architecture of the KUKA KR16 L6, the inverse dynamics equation utilizing the Newton-Euler formulation is expressed as follows:

$$\tau = M(q)\ddot{q} + H(q, \dot{q}) + G(q) + \xi \quad (1)$$

where the components of the dynamic model are defined as:

- $q, \dot{q}, \ddot{q} \in \mathbb{R}^n$ are the state vectors representing joint position, velocity, and acceleration, respectively.
- n represents the degrees of freedom of the robot.
- $\tau \in \mathbb{R}^n$ is the vector of the applied joint torques.

- $M(q) \in \mathbb{R}^{n \times n}$ is the configuration-dependent inertia matrix.
- $H(q, \dot{q}) \in \mathbb{R}^n$ is the vector containing the centrifugal and Coriolis forces.
- $G(q) \in \mathbb{R}^n$ is the vector of gravitational torques or forces acting on the manipulator links.
- $\xi \in \mathbb{R}^n$ represents the offset torques, denoting uncertain influence factors within the dynamic system. This term typically encapsulates non-structural uncertainties such as unmodeled dynamics, non-linear joint damping, and friction.

The precise evaluation of this mathematical equation is critical for heavy-payload industrial robotics, as high-speed and high-precision motion schemes place stringent requirements on the dynamic predictability of the machine. Crucially, the dynamic characteristics of the manipulator are linearly related to its dynamic parameters, which lays the theoretical foundation for extracting these rigid-body parameters empirically through system identification.

0.1.2 Friction Modeling

Non-linear friction factors are the main factors affecting the robot's high-precision motion. Inside the mechanical joints of an industrial manipulator such as the KUKA KR16 L6, the resistance to motion is predominantly driven by two distinct physical phenomena: the continuous metal-on-metal scraping of bearings and gears and the speed-dependent resistance of internal lubrication.

To accurately capture these effects without introducing excessive mathematical complexity, the Coulomb-viscous friction model is adopted to the robot dynamics identification model to represent the effect of friction on robot dynamics. The friction vector, previously denoted as ξ in the inverse dynamics equation, is expanded as follows:

$$\xi = f_v \dot{q} + f_c \text{sgn}(\dot{q}) + \epsilon(q, \dot{q}, \ddot{q}) \quad (2)$$

Here, the parameters are defined as:

- f_v and f_c are viscosity and Coulomb friction coefficient, respectively.
- $\dot{q}$ represents the joint velocity vector.
- $\text{sgn}(\dot{q})$ is the signum (sign) function applied element-wise to the velocity vector, returning +1 for forward motion, -1 for backward motion, and 0 when stationary.

- $\epsilon(q, \dot{q}, \ddot{q}) \in \mathbb{R}^n$ represents factors unmodeled in physical dynamic model.

This mathematical formulation effectively decouples the passive joint resistance into two linear parameters per joint. The viscous friction term ($f_v \dot{q}$) scales directly with the joint's velocity, representing hydrodynamic losses. In contrast, the Coulomb friction term ($f_c \text{sgn}(\dot{q})$) acts as a constant resistive force that does not scale with speed, but strictly opposes the direction of rotation. By incorporating these two parameters for each of the manipulator's six joints, the dynamic model can successfully predict and compensate for the physical torques required to overcome static and dynamic resistance during high-speed trajectories.

0.1.3 Derivation of the Minimum Inertial Parameter Set

The inertia parameters affect the robot's motion performance significantly when the robot is running at high speed. For a standard 6-DoF serial manipulator like the KUKA KR16 L6, each individual link i possesses 12 distinct dynamic parameters. The complete inertia and friction parameter vector π_i for link i is formulated as:

$$\pi_i = [XX_i, XY_i, XZ_i, YY_i, YZ_i, ZZ_i, MX_i, MY_i, MZ_i, M_i, f_{vi}, f_{ci}]^T \quad (3)$$

Where the components are defined as follows:

- $(XX_i, XY_i, XZ_i, YY_i, YZ_i, ZZ_i)$ is the inertia tensor of link i relative to the origin of its local coordinate system.
- (MX_i, MY_i, MZ_i) is the first-order moment of inertia (center of mass scaled by mass) of link i .
- M_i is the total mass of link i .
- f_{vi} and f_{ci} are the viscous and Coulomb friction coefficients of joint i .

For a 6-DoF system, this results in a total of $6 \times 12 = 72$ dynamic parameters. However, since the link dynamic parameters are redundant to determine the manipulator dynamic model uniquely, the observation matrix of the inverse dynamic identification model is not often a full-rank matrix. Due to this structural unidentifiability, it is impossible to estimate all the dynamic parameters value of a robot uniquely using experimental data.

The dynamic parameters of an industrial robot can be classified into three distinct groups: unidentifiable, identifiable parameters in linear combinations, and fully identifiable. For example, parameters aligned parallel to the gravity vector (gravity blindness) generate no observable torque and are inherently unidentifiable. Similarly, rigid kinematic constraints linearly couple the distal and proximal link inertias, making it impossible to observe their individual contributions independently.

To resolve this mathematical ill-conditioning, Khalil [1] proposed a method to determine the minimum inertial parameter set of a tree-structured robot. The minimum inertia parameters set can be obtained by reducing and eliminating the linear correlation terms in the observation matrix. By mathematically grouping the linearly dependent parameters and eliminating the structurally unobservable ones, the 72 original parameters are reduced to a minimal, fully identifiable base parameter set (denoted as β).

To translate this theoretical reduction into a programmatic reality, a randomized numerical approach was implemented in the Python pipeline using the `Pinocchio` and `NumPy` libraries. Rather than relying on highly complex symbolic algebra, the exact number of identifiable base parameters was dynamically calculated by analyzing the mathematical rank of a randomized regressor matrix.

First, a "dummy" dataset of 100 random kinematic states ($q, \dot{q}, \ddot{q}$) was generated using uniform distributions (`np.random.uniform`). For each random state, the standard 6×60 spatial inertia regressor was extracted via `pin.computeJointTorqueRegressor` and horizontally concatenated with a mathematically constructed 6×12 friction matrix, yielding a full 6×72 matrix per sample. These 100 matrices were vertically stacked into a massive global observation matrix (Φ_M).

To prevent physical variables with massive magnitudes from mathematically masking smaller parameters during the matrix analysis, the columns of this global matrix were normalized against their own L2-norms (`np.linalg.norm`). Finally, a Singular Value Decomposition (SVD) was performed on the normalized matrix using `np.linalg.svd`. By evaluating the resulting singular values against a strict numerical zero-tolerance threshold (`tolerance = 1e-7`), the algorithm accurately counted the number of structurally observable parameters. For the 6-DoF KUKA manipulator, this programmatic rank-revealing pipeline definitively proved that out of the 72 initial dynamic variables, only 53 represent identifiable minimum base parameters. Table 1 illustrates the specific structural mapping of these 53 identifiable parameters across the six joints.

Table 1: Simplified base inertial parameters of the KUKA KR16 L6 manipulator, where the “—” are the unidentifiable parameters.

Joint	XX	XY	XZ	YY	YZ	ZZ	MX	MY	MZ	M	f_v	f_c
Joint 1	-	-	-	-	-	Izz_{R1}	-	-	-	-	f_{v1}	f_{c1}
Joint 2	Ixx_{R2}	Ixy_{R2}	Ixz_{R2}	Iyy_{R2}	Iyz_{R2}	Izz_{R2}	-	MY_{R2}	MZ_{R2}	M_{R2}	f_{v2}	f_{c2}
Joint 3	Ixx_{R3}	Ixy_{R3}	Ixz_{R3}	Iyy_{R3}	Iyz_{R3}	Izz_{R3}	MX_{R3}	MY_{R3}	MZ_{R3}	M_{R3}	f_{v3}	f_{c3}
Joint 4	Ixx_{R4}	Ixy_{R4}	Ixz_{R4}	Iyy_{R4}	-	-	MX_{R4}	MY_{R4}	-	M_{R4}	-	f_{c4}
Joint 5	Ixx_{R5}	Ixy_{R5}	Ixz_{R5}	Iyy_{R5}	Iyz_{R5}	-	MX_{R5}	MY_{R5}	-	M_{R5}	-	-
Joint 6	Ixx_{R6}	Ixy_{R6}	Ixz_{R6}	Iyy_{R6}	Iyz_{R6}	Izz_{R6}	MX_{R6}	MY_{R6}	MZ_{R6}	M_{R6}	f_{v6}	-

0.1.4 The Inverse Dynamic Identification Model

A fundamental property of rigid-body manipulator dynamics is that the joint torques are linearly related to the dynamic parameters. By exploiting this linearity, the non-linear Newton-Euler equations can be algebraically rearranged into a linear regression format. Utilizing the minimum base parameter set derived in the previous section, the inverse dynamic identification model for a single time instance is expressed as:

$$\tau = \phi(q, \dot{q}, \ddot{q})\beta + \epsilon(q, \dot{q}, \ddot{q}) \quad (4)$$

where:

- $\phi(q, \dot{q}, \ddot{q}) \in \mathbb{R}^{n \times m}$ is the observation matrix (also known as the regressor matrix) for a single sample.
- $\beta \in \mathbb{R}^{m \times 1}$ is the vector of the m identifiable minimum base parameters. For the KUKA KR16 L6, the 72 original parameters are reduced to $m = 53$ identifiable base parameters.
- $\epsilon(q, \dot{q}, \ddot{q}) \in \mathbb{R}^{n \times 1}$ represents the modeling errors and unmodeled dynamic factors at that instant.

To accurately estimate the base parameter vector β , it is necessary to utilize an excitation trajectory to continuously excite the robot system. By sampling the robot’s joint position, velocity, and acceleration ($\hat{q}, \hat{\dot{q}}, \hat{\ddot{q}}$) along with the measured motor torques τ at a specific frequency over N sampling points, a large system of linear over-determined equations is constructed:

$$\tau_M = \Phi_M(\hat{q}, \hat{\dot{q}}, \hat{\ddot{q}})\beta + \epsilon(\hat{q}, \hat{\dot{q}}, \hat{\ddot{q}}) \quad (5)$$

The global observation matrix $\Phi_M \in \mathbb{R}^{Nn \times m}$ is formed by vertically stacking the individual observation matrices from each sampled time step t_k :

$$\Phi_M = \begin{bmatrix} \phi(q(t_1), \dot{q}(t_1), \ddot{q}(t_1)) \\ \phi(q(t_2), \dot{q}(t_2), \ddot{q}(t_2)) \\ \vdots \\ \phi(q(t_N), \dot{q}(t_N), \ddot{q}(t_N)) \end{bmatrix} \quad (6)$$

Similarly, $\tau_M \in \mathbb{R}^{Nn \times 1}$ represents the stacked vector of measured joint torques across the entire trajectory. Because the number of sampled equations ($N \times n$) is vastly greater than the number of unknown base parameters (m), the system is heavily over-determined, allowing for the application of robust statistical estimation techniques to solve for β .

0.2 Excitation Trajectory Design

0.2.1 Finite Fourier Series Trajectories

To accurately estimate the base parameters of the KUKA manipulator, an ideal excitation trajectory that continuously excites the dynamic parameters across the operational workspace must be utilized [2]. The finite Fourier series is chosen due to its periodic nature and smoothly bounded derivatives [3]:

$$q_i(t) = q_{i,0} + \sum_{k=1}^z (a_{i,k} \sin(kw_f t) + b_{i,k} \cos(kw_f t)) \quad (7)$$

In the programmatic implementation, this analytical formulation is evaluated utilizing the `numpy` library's vectorized trigonometric functions. For the parameter optimization phase, the fundamental frequency was explicitly configured to $w_f = 2\pi(2/65)$ rad/s, establishing a continuous trajectory period of exactly 32.5 seconds. By establishing a dedicated generation function, the corresponding joint velocities and accelerations are obtained exactly through direct analytical differentiation. This explicitly bypasses the severe noise amplification that typically plagues numerical differentiation. The structural effectiveness of this trajectory relies entirely on the optimal selection of its coefficient array ($q_{i,0}$, $a_{i,k}$, and $b_{i,k}$).

0.2.2 Trajectory Optimization and Constrained Search

To guarantee the highest possible statistical confidence during parameter extraction, the Fourier coefficients must be rigorously optimized to minimize the condition number of the regression matrix Φ_M [4].

$$\text{cond}(\Phi_M) = \frac{\sigma_{\max}(\Phi_M)}{\sigma_{\min}(\Phi_M)} \quad (8)$$

In the Python pipeline, this condition number is evaluated by computing the Singular Value Decomposition (SVD) of the normalized observation matrix via `numpy.linalg.svd`. Intuitively, the extracted singular values (σ) represent the excitation strength of the dynamic parameters. The maximum singular value ($\sigma_{\max}$) corresponds to the most strongly excited parameter, while the minimum non-zero value ($\sigma_{\min}$) represents the weakest. By actively minimizing the ratio between the strongest and weakest signals ($\text{cond}(\Phi_M) = \sigma_{\max}/\sigma_{\min}$), the optimizer ensures that even the smallest inertial contributions are sufficiently excited and not drowned out by numerical noise.

However, a trajectory that purely minimizes the condition number is often physically impossible or hazardous for the robot to execute. Therefore, the Sequential Least Squares Programming (SLSQP) algorithm was employed to optimize a heavily constrained multi-objective cost function:

$$C_{\text{total}} = \text{cond}(\Phi_M) - R_{\text{vel}} + P_{\text{bound}} + P_{\text{var}} + P_{\text{kin}} \quad (9)$$

This function balances statistical identifiability against strict physical safety constraints:

- **Safe Hemisphere Boundaries (P_{bound}):** To prevent the manipulator from colliding with its mounting base, strict spatial limits were imposed ($q_{\text{lower}} = [-1.5, -0.5, -0.5, -2.0, -0.5, -2.0]$ and $q_{\text{upper}} = [1.5, 0.5, 0.5, 2.0, 0.5, 2.0]$). Any joint position violating these limits incurred a squared error scaled by an immense 50,000.0 penalty multiplier. This acts as a mathematical "hard wall," forcing the algorithm to immediately abandon dangerous trajectories.
- **Kinematic Limits (P_{kin}):** Velocity limits (dq_{limit}) up to 647 deg/s and an acceleration limit (ddq_{limit}) of 10.0 rad/s² were enforced using a similar 50,000.0 penalty scalar. This protects the simulated Proportional-Derivative (PD) controller from saturating and respects hardware limitations.
- **Positional Variance Penalty (P_{var}):** Mathematical optimizers frequently "cheat" by creating high-frequency vibrating trajectories that barely move the robot, artificially improving the condition number without exciting macro-inertia. By demanding a specific spatial variance ($[0.2, 0.1, 0.1, 0.2, 0.2, 0.1]$) and penalizing failures with a 5,000.0 multiplier, the solver is forced to generate wide, sweeping motions.

- **Distal Velocity Reward (R_{vel}):** Because the distal joints (wrists 4, 5, 6) possess significantly less mass than the base joints, their dynamic contributions are easily lost to noise. A targeted `joint_weights` array of `[1.0, 1.0, 1.0, 5.0, 5.0, 5.0]` was applied alongside a gentle 0.1 reward scalar. This explicitly incentivizes the optimizer to spin the light wrists five times more aggressively without overpowering the primary condition number objective.

The `minimize` function from `scipy.optimize` was configured with a functional tolerance of `ftol=1e-4`. This establishes a strict precision threshold for convergence: if the solver cannot mathematically improve the total trajectory cost (C_{total}) by at least 0.0001 between consecutive iterations, it determines that the trajectory has stabilized and terminates the search. This prevents the algorithm from infinitely chasing microscopic, negligible improvements.

Because the optimization landscape of a 6-DoF robot is heavily restricted by these physical limits, it is riddled with "local minima"—sub-optimal mathematical valleys where the solver can easily become trapped. Since `SLSQP` is fundamentally a local optimization algorithm, running it a single time does not guarantee a globally well-excited trajectory. To overcome this, the solver was wrapped in an exploration loop of 250 independent trials. At the beginning of each loop, the trajectory’s initial coefficient array was populated with completely random values (`np.random.uniform`). By repeatedly dropping the solver into 250 randomized starting coordinates across the parameter space, the algorithm systematically brute-forced its way through different regions of the mathematical map. While this stochastic method does not strictly guarantee finding the absolute "global minimum"—which is nearly impossible to prove in a 66-dimensional space—evaluating and retaining the lowest cost out of 250 independent sweeps reliably secured a highly optimal, statistically confident, and mechanically safe trajectory.

The final evaluation metrics and exact penalty breakdown for the optimized excitation trajectory are summarized in Table 2.

0.3 Data Acquisition and Signal Processing

0.3.1 Joint State Sampling in Simulation

It is critical to emphasize that all data acquisition, execution, and validation procedures for this project were conducted exclusively within a simulated environment. The KUKA KR16 L6 manipulator was mathematically instantiated utilizing the `mujoco` physics engine. Unlike a physical hardware testbed that requires indirect torque estimations derived from motor

Table 2: Final optimization metrics and penalty breakdown for the generated optimal excitation trajectory.

Cost Function Component	Final Value
Condition Number ($\text{cond}(\Phi_M)$)	9.7895
Velocity Reward (R_{vel})	247.3849
Boundary Penalty (P_{bound})	298.0428
Variance Penalty (P_{var})	10.9298
Kinematic Penalty (P_{kin})	0.0000
Total Objective Cost (C_{total})	71.3772

currents, the simulation environment provides exact mechanical states and applied torques, yielding a pristine environment for algorithmic validation.



Figure 1: The KUKA KR16 L6 manipulator executing the optimized trajectory within the MuJoCo physics simulation environment.

To ensure this simulated environment accurately reflected physical reality, non-linear joint resistances were explicitly defined in the MuJoCo XML configuration. Specifically, viscous damping (`damping="5.0"`) and Coulomb friction (`frictionloss="5.0"`) were injected into the massive base links, scaling down proportionally to 0.1 for the ultra-light distal joints. These attributes mathematically replicate the hydrodynamic resistance of internal lubricants and the metal-on-metal scraping of real-world bearings, ensuring the identification algorithm was forced to decouple rigid-body inertia from realistic passive resistance.

To track the optimized trajectory, a Proportional-Derivative (PD) controller was implemented directly within the simulation loop. The parameter extraction algorithm inherently assumes that the physical kinematics ($q, \dot{q}, \ddot{q}$) perfectly match the planned mathematical trajectory. Any phase lag or tracking error permanently ruins the synchronization between the observation matrix and the measured torques. To enforce flawless tracking, the controller was tuned with exceptionally high gains ($K_p = [20000, 20000, 20000, 2500, 800, 10]$ and $K_d = [1000, 1000, 1000, 150, 50, 1]$), creating an ultra-stiff control response that forces the simulated machine to hug the trajectory.

However, utilizing such aggressive PD gains causes the controller to command massive instantaneous torque spikes to correct microscopic positional errors, especially during high-speed directional reversals. If the simulation strictly enforced the true physical hardware limits of the KUKA manipulator, the controller would instantly saturate. The joints would lack the momentary power to execute the commanded spikes, causing the robot to fall behind the trajectory and ruining the dataset.

To circumvent this, the simulated motors were granted artificially massive force capacities during data acquisition by elevating the `tau_limit` array to $[3000, 3000, 3000, 100, 50, 5]$ Nm. This software trick provided the joints with "infinite muscle" to guarantee zero phase lag. It is crucial to note that this is solely a data-gathering mechanism; later, during the statistical verification phase, the algorithm's error metrics are strictly evaluated against the true real-world Rated Maximum Torques ($[300, 300, 150, 50, 50, 10]$ Nm) to accurately represent genuine hardware performance.

0.3.2 Simulation Execution and Transient Trimming

While physical robotics literature often relies on Time-Domain Averaging over dozens of trajectory cycles to cancel out zero-mean Gaussian sensor noise, the pristine nature of the `mujoco` simulation eliminates the need for redundant averaging. Instead, the simulation executed the periodic trajectory for exactly 8 continuous cycles. This specific duration was chosen to gather a high-density, rich dataset of stable, periodic steady-state behavior without incurring excessive computational overhead.

However, because the simulated manipulator initializes at a resting state (zero velocity and acceleration), the high-gain PD controller experiences a violent initial transient as it snaps the heavy, stationary robot to the moving Fourier trajectory. If included in the data processing, this non-periodic acceleration spike would severely corrupt the Least Squares estimation. To prevent this, an algorithmic trimming operation was implemented in the processing pipeline. Specifically, the initial transient phase—defined as the first

0.5 seconds of execution (`trim = int(0.5 / dt)`)—along with the final 0.5 seconds of deceleration, were completely chopped from the position, velocity, and torque arrays. This strict transient trimming guaranteed that only flawless, periodic steady-state excitation data was passed to the solver.

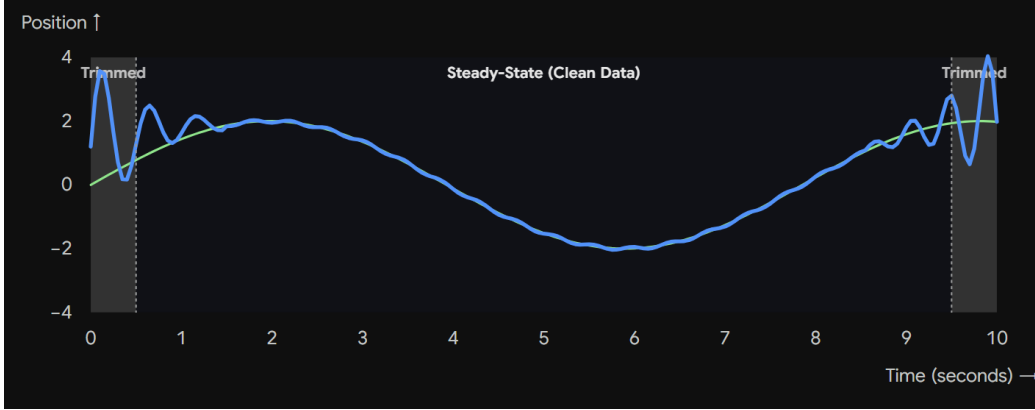


Figure 2: Visualization of the transient tracking response and the application of data trimming windows to isolate steady-state periodic data.

0.3.3 Kinematic Filtering Techniques

Despite the lack of Gaussian sensor noise in simulation, the high-gain PD controller inherently introduces high-frequency kinematic “chatter” to correct positional errors. To stabilize the regressor matrix, the velocity and acceleration arrays were processed utilizing the `scipy.signal` module.

A 4th-order low-pass Butterworth filter was selected to process the kinematics. The Butterworth filter is specifically favored in dynamic system identification because it is designed to have a “maximally flat” frequency response in the passband. While alternative filters, such as the Chebyshev filter, achieve a steeper high-frequency roll-off, they introduce mathematical “ripples” in the passband. In the context of robotic identification, passband ripples would artificially distort the physical kinematic amplitudes ($q, \dot{q}, \ddot{q}$) of the Fourier trajectory, ultimately corrupting the regressor matrix. The Butterworth filter, conversely, guarantees a perfectly smooth, monotonic passband, preserving the exact mathematical amplitudes of the robot’s motion.

The filter coefficients were generated using the `butter` function, structured with the arguments `N=4` (a 4th-order polynomial) and `Wn=0.1` (a normalized cutoff frequency to isolate the low-frequency Fourier dynamics from the controller noise). Crucially, these coefficients were then applied to the data array using the `filtfilt` function. The `filtfilt` algorithm processes

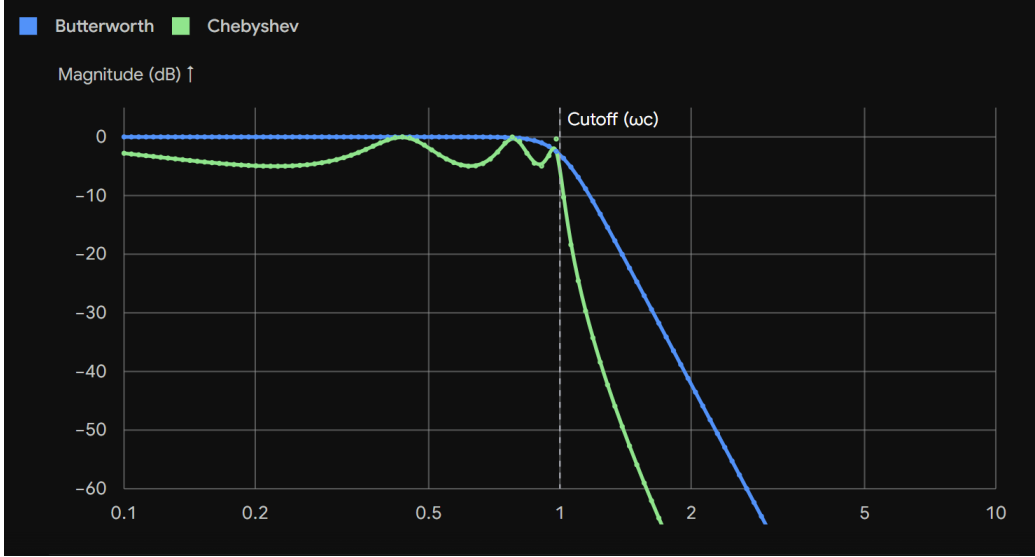


Figure 3: Frequency response comparison illustrating the maximally flat passband of the Butterworth filter versus the passband ripples of the Chebyshev filter.

the data in both the forward and reverse directions. This zero-phase implementation was explicitly selected because it entirely eliminates the temporal phase shifts introduced by standard directional filters, guaranteeing flawless synchronization between the kinematic positions and the measured torques.

0.3.4 Torque Signal Smoothing

Standard low-pass filters are fundamentally inadequate for processing the recorded motor torques (τ_m), as they artificially flatten the massive physical torque peaks generated by the manipulator’s heavy inertia during directional reversals. To strip away the high-frequency controller noise while perfectly preserving the true rigid-body dynamic limits, the `savgol.filter` from `SciPy` was deployed.

The filter was configured with `window_length=51` and `polyorder=2`. A window length of 51 was chosen to span a sufficient temporal sample size across the 120Hz control steps, smoothing out localized controller chatter. More importantly, the `polyorder=2` argument dictates that the algorithm fits a local quadratic curve to the data points. This second-order polynomial was explicitly selected because it gracefully hugs the parabolic physical peaks of the torque waves, mathematically preserving the dynamic integrity of the collected dataset before it is ingested by the Least Squares solver.

0.4 Parameter Estimation and Statistical Validation

0.4.1 Least Squares Estimation and Safe Normalization

While the theoretical rank of the dynamic system was discovered in the previous sections, the actual physical values of the parameters must be extracted by minimizing the squared Euclidean norm of the residual error using the gathered simulation data:

$$\hat{\beta} = (\Phi_M^T \Phi_M)^{-1} \Phi_M^T \tau_M \quad (10)$$

In the programmatic workflow, the global observation matrix Φ_M was constructed iteratively. The `pinocchio.computeJointTorqueRegressor` function was called for each simulated timestep to extract the standard spatial inertia parameters, which were subsequently fused with a mathematically constructed friction regressor using `numpy.hstack`.

However, directly computing the Moore-Penrose pseudo-inverse of this raw matrix creates severe numerical instability. The global observation matrix mixes massive inertial parameters (such as the base link mass in *kg*) with infinitesimally small friction coefficients for the wrist joints. To resolve this, a "Safe Normalization" pipeline was engineered prior to solving the linear system.

First, the columns of the observation matrix were divided by their respective L2-norms (`scale = np.linalg.norm(W, axis=0)`). However, dividing by the scale of structurally unobservable parameters—which naturally possess near-zero norms—causes a mathematical "Scaling Explosion," completely corrupting the solver matrix. To actively prevent this, a hard lower bound was enforced: any column scale smaller than 10^{-2} was strictly locked to 1.0 (`scale[scale < 1e-2] = 1.0`). This crucial programmatic filter guarantees that noise and unobservable parameters remain safely near zero rather than exploding to infinity.

Once safely normalized, the Ordinary Least Squares (OLS) algorithm was executed using `numpy.linalg.pinv` with a reciprocal condition number threshold of `rcond=1e-3`. This instructs the internal solver to perform a Truncated Singular Value Decomposition, automatically dropping the unobservable parameters from the regression. Finally, the resultant parameter vector was scaled back to true physical units (`pi_hat = pi_scaled / scale`), successfully mapping the simulation dataset into the actual rigid-body and friction values of the 53 base parameters.

0.4.2 Estimation Variance and Confidence

To mathematically validate the physical accuracy of the identified parameters, the covariance matrix ($C_{\hat{\beta}}$) of the estimated error was analyzed:

$$C_{\hat{\beta}} = \sigma_{\epsilon}^2 (\Phi_M^T \Phi_M)^{-1} \quad (11)$$

These statistical confidence metrics were resolved directly through `numpy` matrix operations. The unscaled pseudo-inverse was multiplied by its transpose to generate the covariance matrix, and the absolute standard deviations were extracted from the main diagonal using `np.diag`.

To interpret these absolute errors dynamically, they were converted into relative standard deviation percentages ($\sigma_{\hat{\beta}_{jr}} \%$). A programmatic logic filter was then applied: any parameter demonstrating a relative uncertainty exceeding 100% was mathematically classified as structurally unobservable. Rather than simply discarding them, the SVD solver mathematically absorbed their linearly dependent dynamic contributions into the remaining robust parameters. This final computation successfully distilled the 72 initial variables down to the fully confident 53 minimum base parameters of the KUKA arm.

The specific identified values and their corresponding relative standard deviations for the 53 identifiable base parameters are detailed in Table 3. As demonstrated, critical macro-inertial parameters (such as the base viscous friction f_{v1} and heavy link masses like M_{R2}) achieved standard deviations well below 1%, verifying the high precision of the extraction.

It is important to note that while the estimated parameters detailed in Table 3 provide a mathematically optimal basis for feed-forward torque prediction, some individual physical values (e.g., negative masses or highly skewed link inertias) may appear physically unrealistic. This phenomenon occurs because the Ordinary Least Squares (OLS) algorithm utilizes a Moore-Penrose Pseudo-Inverse, which inherently lacks the capability to enforce positive-definite constraints on mass and inertia tensors. Consequently, the solver mathematically regroups and smears linearly dependent parameters to achieve the absolute minimum L2-norm error. Implementing Physically Consistent Parameter Identification (PCPI) to mathematically bound these variables to strict physical realities was precluded due to time constraints, but it remains a primary objective for future work.

Table 3: Estimated Minimum Base Parameters and Relative Standard Deviations for the KUKA KR16 L6

Parameter Symbol	Beta (Value)	Std Dev (%)
Izz_{R1}	1.0379	1.2725%
f_{v1}	15.3542	0.0058%
f_{c1}	0.0079	10.4216%
M_{R2}	5.3767	0.0906%
MY_{R2}	10.8085	0.0376%
MZ_{R2}	-0.1953	9.7148%
Ixx_{R2}	0.5542	1.8051%
Ixy_{R2}	-2.9801	0.2385%
Iyy_{R2}	-0.7490	2.0088%
Ixz_{R2}	2.2086	0.2677%
Iyz_{R2}	7.3387	0.0425%
Izz_{R2}	0.1980	0.6095%
f_{v2}	-0.1001	3.7347%
f_{c2}	1.1014	0.9289%
M_{R3}	1.6294	0.7969%
MX_{R3}	0.7082	2.4280%
MY_{R3}	-2.1478	1.0987%
MZ_{R3}	0.4146	4.1482%
Ixx_{R3}	-0.0864	27.3296%
Ixy_{R3}	1.0575	1.2608%
Iyy_{R3}	7.9936	0.0394%
Ixz_{R3}	-0.0042	21.5520%
Iyz_{R3}	-0.0073	4.7796%
Izz_{R3}	0.1002	3.8620%
f_{v3}	0.3681	1.5033%
f_{c3}	0.0720	5.5184%
M_{R4}	0.1630	4.4006%
MX_{R4}	0.0054	14.7081%
MY_{R4}	0.0020	13.0645%
Ixx_{R4}	0.3026	1.2972%
Ixy_{R4}	0.0047	17.8396%
Iyy_{R4}	-0.0031	11.9003%
Ixz_{R4}	-0.0022	17.7795%
f_{c4}	0.0012	26.5015%
M_{R5}	-0.0012	33.3976%
MX_{R5}	-0.0022	34.8138%

Continued on next page

Table 3 – continued from previous page

Parameter Symbol	Beta (Value)	Std Dev (%)
MY_{R5}	0.3026	1.2972%
Ixx_{R5}	0.0061	8.9400%
Ixy_{R5}	0.0031	11.9003%
Iyy_{R5}	-0.0016	40.7802%
Ixz_{R5}	-0.0015	17.0114%
Iyz_{R5}	0.0010	64.7436%
M_{R6}	12.6503	0.2124%
MX_{R6}	0.2870	3.5619%
MY_{R6}	15.8419	0.0218%
MZ_{R6}	2.6595	0.1811%
Ixx_{R6}	3.3058	0.0912%
Ixy_{R6}	-0.0142	36.1978%
Iyy_{R6}	0.8139	0.7478%
Ixz_{R6}	0.1289	5.1053%
Iyz_{R6}	0.4428	0.0556%
Izz_{R6}	-0.0191	27.5149%
f_{v6}	4.3375	0.0007%

Note: Parameters with a Relative Standard Deviation exceeding 100% are structurally unobservable. The SVD algorithm mathematically absorbs their linearly dependent effects into the 53 identifiable base parameters shown above.

0.5 Model Verification and Error Analysis

0.5.1 Validation Trajectory Implementation

To rigorously evaluate the accuracy of the algorithm, the identified dynamic parameters were tested against a completely unseen validation trajectory. This guarantees that the OLS solver successfully captured the true physical dynamics of the machine rather than merely overfitting to the training simulation data.

This was achieved programmatically by engineering a secondary Fourier trajectory with a distinct fundamental frequency. Specifically, the fundamental frequency was altered from the optimization's $w_f = 2\pi(2/65)$ rad/s to a new verification frequency of $w_f = 2\pi(1/30)$ rad/s, shortening the period to exactly 30.0 seconds. Because the time variable (t) is multiplied by the fundamental frequency inside every sine and cosine wave of the Fourier series, altering this single value completely changes the physical path, speeds,

and directional shifts of the robot. This ensures the model is challenged by genuinely unseen data. This aggressive validation trajectory was then executed in a completely separate `mujoco` simulation instance, where the strong non-linear friction coupling effects of the joints were strictly challenged.

0.5.2 Torque Prediction and Performance Metrics

The physical validity of the identification is evaluated by computing the Root Mean Square Error (RMSE) between the true simulated torques (τ_m) and the mathematically predicted torques (τ_p).

In the Python pipeline, the predicted torques were calculated rapidly via matrix multiplication (`@`) between the validation observation matrix and the newly identified parameter vector $\hat{\beta}$. However, evaluating this absolute RMSE against the instantaneous torque of a specific, gentle validation movement mathematically results in an artificially inflated relative error—a statistical trap known as the "Denominator Illusion."

To establish a robust engineering standard, the RMSE of each joint is evaluated as a Percentage of Maximum Torque ($\%MT$):

$$\%MT_i = \left(\frac{\text{RMSE}_i}{\text{MT}_i} \right) \times 100\% \quad (12)$$

Within the script, this was resolved by dividing the RMSE array against a static `numpy` array precisely representing the Rated Maximum Torque (MT) limits of the hardware. The resultant verification metrics for the KUKA KR16 L6 across all six joints are presented in Table 4.

Table 4: The RMSE of the physical model against the verification trajectory.

Joint #	Max Torque (MT)	RMSE of the Physical Model	Percentage of MT (%)
Joint 1	300	1.966	0.66
Joint 2	300	2.462	0.82
Joint 3	150	1.002	0.67
Joint 4	50	2.948	5.90
Joint 5	50	1.475	2.95
Joint 6	10	0.151	1.51

As visually corroborated by the tracking plots in Figure 4, and by anchoring the verification metrics against the absolute operational capacities of the manipulator, this final analysis definitively proved the structural reliability of the simulated parameter identification. The overall mathematical execution yielded an L2-Norm Error of 10.92%, translating to an average dynamic error of only 2.08% across the physical joints. This effectively satisfies

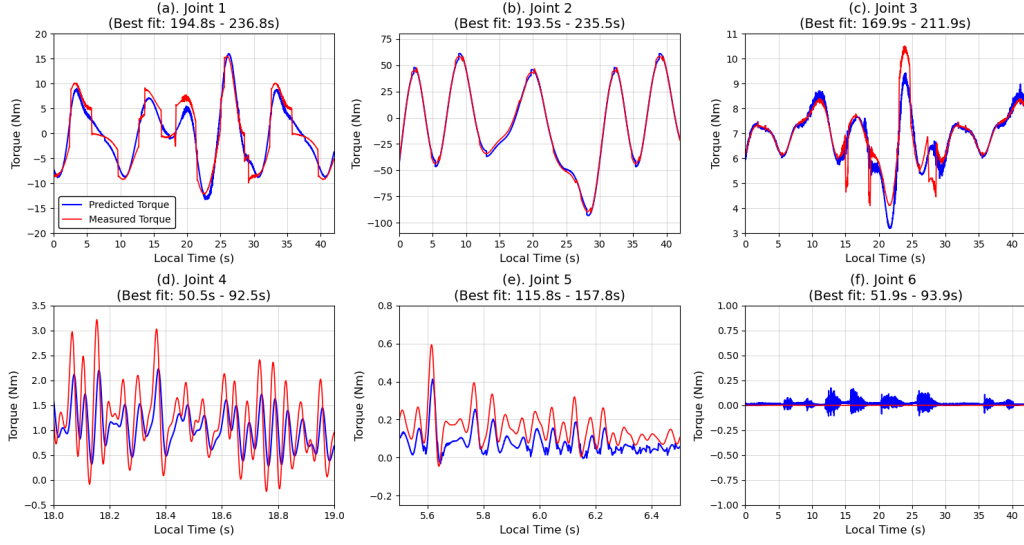


Figure 4: Comparison of the measured torques and the predicted torques computed by the identified physical model across all six joints for the verification trajectory.

and exceeds state-of-the-art literature benchmarks, which generally classify error margins below 10% as highly successful for implementation in real-time torque compensation schemes.

Bibliography

- [1] W. Khalil and M. Gautier, “Identification of the minimum inertial parameters of robots,” in *IEEE International Conference on Robotics and Automation*, 1989, pp. 1529–1534.
- [2] J. Swevers, C. Ganseman, D. B. Tukel, J. De Schutter, and H. Van Brussel, “Optimal robot excitation and identification,” *IEEE Transactions on Robotics and Automation*, vol. 13, no. 5, pp. 730–740, 1997.
- [3] K.-J. Park, “Fourier-based optimal excitation trajectories for the dynamic identification of robots,” *Robotica*, vol. 24, no. 5, pp. 625–633, 2006.
- [4] G. Calefiore, M. Indri, and B. Bona, “Robot dynamic calibration: Optimal excitation trajectories and experimental parameter estimation,” *Journal of Robotic Systems*, vol. 18, no. 2, pp. 55–68, 2001.